

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Институт компьютерных наук

Кафедра автоматизированных систем управления

Практическая работа №6

по дисциплине

«Управление ИТ-проектами»

Студент М-РИТ-25-1 _____ Станиславчук С.М.

Руководитель

К.т.н., доцент _____ Ведищев В.В.

Липецк, 2026

Содержание

1. Планирование версий программного продукта 3
   1.1 Фазы процесса разработки 3
   1.2 Планирование и построение дерева версий 3
   1.3 Средства контроля версий 4

1. Планирование версий программного продукта

1.1. Фазы процесса разработки

Процесс разработки программного продукта разбивается на последовательные фазы, каждая из которых соответствует определённому состоянию системы. На начальном этапе осуществляется формирование требований и проектирование архитектуры. Далее следует фаза активной разработки, в рамках которой реализуется основной функционал системы.

После реализации ключевых компонентов начинается этап тестирования, направленный на выявление ошибок и проверку соответствия системы требованиям. Завершающей фазой является выпуск стабильной версии продукта и его последующа1я эксплуатация с возможностью внесения доработок и исправлений.

Каждая фаза сопровождается выпуском промежуточных версий, что позволяет контролировать прогресс разработки и своевременно выявлять проблемы.

1.2. Планирование и построение дерева версий

Для управления развитием системы используется схема версионирования вида X.Y.Z, где каждая компонента отражает уровень изменений в продукте.

Мажорная версия (X.Y.Z) изменяется в случае значительных изменений системы. К таким изменениям относятся переработка архитектуры, внедрение новых ключевых функций или изменение существующих интерфейсов, несовместимое с предыдущими версиями. Выпуск мажорной версии означает качественный переход системы на новый уровень.

Минорная версия (X.Y.Z) увеличивается при добавлении нового функционала без нарушения совместимости с предыдущими версиями. Это могут быть новые возможности системы, расширение существующих функций или улучшения, не влияющие на основную архитектуру.

Патч-версия (X.Y.Z) увеличивается при внесении незначительных изменений, таких как исправление ошибок, оптимизация производительности или незначительные доработки интерфейса. Такие изменения не затрагивают функциональность системы с точки зрения пользователя.

Дерево версий реализуется через систему ветвления. Основной веткой является `main`, содержащая стабильную версию системы. Параллельно используется ветка `develop`, в которой происходит интеграция новых функций перед выпуском релиза.

Функциональность разрабатывается в отдельных ветках, создаваемых от `develop`. Например:

- ветки вида `feature/chat` — реализация текстовых сообщений;
- `feature/voice` — реализация голосовой связи (UDP, RTP);
- `feature/file-transfer` — передача файлов;
- `feature/auth` — аутентификация и управление пользователями;
- `feature/ui` — разработка пользовательского интерфейса.

Исправления ошибок выполняются в ветках типа `bugfix/*`, которые также создаются от `develop` и после проверки сливаются обратно.

Для критических исправлений в уже выпущенных версиях используются ветки `hotfix/*`, создаваемые от `main`. После исправления изменения интегрируются как в `main`, так и в `develop`.

Перед выпуском версии формируется ветка `release/X.Y.0`, в которой выполняется финальное тестирование и подготовка релиза. После этого ветка сливается в `main`, и версия помечается тегом.

1.3. Средства контроля версий

В качестве системы контроля версий используется Git, при котором для каждой роли или задачи создаются отдельные ветки. Это позволяет изолировать изменения и упростить процесс их интеграции.

Основная ветка содержит стабильную версию продукта, в то время как разработка нового функционала ведётся в отдельных ветках. После завершения работы изменения проходят проверку и объединяются с основной веткой.

Для обозначения стабильных релизов используются теги, соответствующие номеру версии. Это позволяет фиксировать состояние системы на момент выпуска и упрощает возврат к предыдущим версиям при необходимости.

Ответственность за управление версиями и выпуск релизов возлагается на бэкенд-разработчика, который контролирует целостность системы, корректность версионирования и процесс интеграции изменений.